TP 4 Projet Fédéré

Automatisation : Test d'acceptation

Objectifs :

- Comprendre le cycle de vie des tests d'acceptation.
- Maîtriser la configuration d'un outil de CI (GitHub Actions).
- Automatiser l'exécution des tests lors de chaque modification de code (**Push**) ou demande de fusion (**Pull Request**)

Partie 1 : Structure du Projet sur GitHub

Une structure professionnelle sur GitHub est une nécessité. Il faut avoir une organisation normalisée des fichiers et dossiers en respectant les conventions du langage et des frameworks utilisés. Une telle structure vise à rendre le projet maintenable, scannable et collaboratif.

Elle repose généralement sur quatre piliers :

- **Séparation des préoccupations :** Distinction claire entre le code source (`src/main`), le code de test (`src/test`) et les ressources de configuration.
- **Standardisation :** Utilisation de noms de dossiers conventionnels (ex: `docs/`, `scripts/`, `bin/`) facilitant l'intégration avec les outils de build (Maven, Gradle, Pytest).
- **Infrastructure-as-Code :** Présence d'un dossier racine pour l'automatisation (ex: `.github/workflows/` pour la CI/CD).
- **Documentation de base :** Inclusion systématique de fichiers standards à la racine :
  - `README.md` (guide d'utilisation).
  - `.gitignore` (exclusion des fichiers temporaires/sensibles).
  - `LICENSE` (cadre juridique).
  - `CONTRIBUTING.md` (règles de collaboration).

Partie 2 : Automatisation avec GitHub Actions (Le Workflow)

1. Créer un fichier `.github/workflows/acceptance-tests.yml`
   - Ce fichier indique à GitHub exactement quand et comment lancer les tests.
   - L'utilisation de `pull_request` est cruciale. Elle permet de bloquer un "Merge" si les tests d'acceptation échouent, protégeant ainsi l'intégrité de la branche principale.
2. Rédaction du scénario (Gherkin)
   - Définir un comportement attendu dans un fichier `.feature`
3. Développement des "Step Definitions"
   - Coder la logique de test (avec Selenium ou Playwright pour le web) qui répond aux phrases Gherkin.

4. Vérification sur GitHub
   - Effectuer un `git push`.
   - Observer le lancement automatique de la suite de tests dans l'onglet **"Actions"** du dépôt GitHub.
5. Critères d'Évaluation
   - Le déclenchement correct : Le test s'est-il bien lancé lors du Push ?
   - L'isolation : Le test passe-t-il dans l'environnement virtuel de GitHub (Ubuntu) ?
   - Le reporting : est-il possible de consulter les logs d'erreur en cas d'échec dans l'interface GitHub ?

Partie 3 : Exemple illustratif

1. Cadre Applicatif : Site de Vente en Ligne

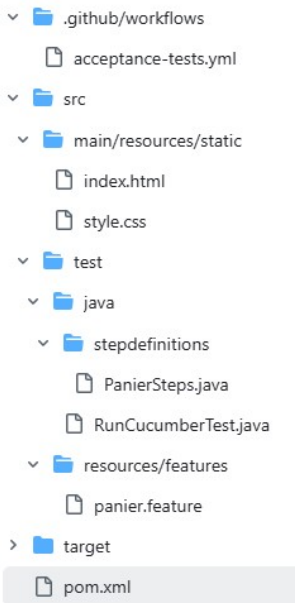
L'objectif est de valider le comportement métier suivant : « *Un utilisateur doit pouvoir ajouter un produit à son panier et voir le compteur se mettre à jour immédiatement* ».

N.B. : Il s'agit d'un **exemple minimaliste** (Proof of Concept). L'accent n'est pas mis sur la complexité du code source, mais sur la **chaîne d'automatisation complète** : de l'écriture du besoin métier à l'exécution automatique sur un serveur distant (GitHub Actions).

2. Structure du projet

Dans le présent TP, le projet respecte la structure d'un projet Java (Maven) qui **s**uit la convention src/main/java et src/test/java.

Structure à créer localement et à ajouter sur GitHub:



Chaque fichier créé joue un rôle précis dans l'architecture de test :

- **`pom.xml` (Le Chef d'Orchestre) :** Il centralise les dépendances (Selenium, Cucumber, JUnit) et configure le cycle de vie du projet. Sans lui, Maven ne sait pas comment compiler ni quels outils utiliser.
- **`panier.feature` (La Spécification) :** Rédigé en Gherkin, ce fichier est le lien entre le client et le développeur. Il décrit le scénario de test en langage naturel (Given/When/Then).
- **`index.html` & `style.css` (Le SUT - System Under Test) :** C'est votre application. Nous séparons la structure du design pour respecter les bonnes pratiques de développement web et permettre à Selenium de cibler des éléments précis via leurs **IDs**.
- **`PanierSteps.java` (Le Moteur) :** Ce fichier traduit chaque phrase du fichier `.feature` en instructions techniques (clics, saisies, vérifications) que le navigateur doit exécuter.
- **`RunCucumberTest.java` (Le Lanceur) :** Il sert de pont entre JUnit et Cucumber. C'est lui qui indique à Maven où se trouvent les scénarios et comment générer les rapports de test.
- **`acceptance-tests.yml` (L'Automatisation CI) :** Situé dans `.github/workflows`, ce fichier définit la recette pour que GitHub crée un serveur virtuel, lance votre site et exécute les tests à chaque fois que vous modifiez votre code.

3. Procédure de mise en œuvre

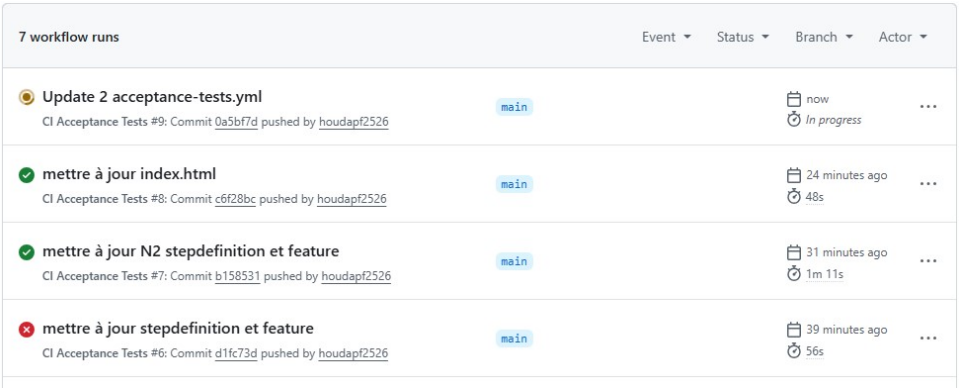
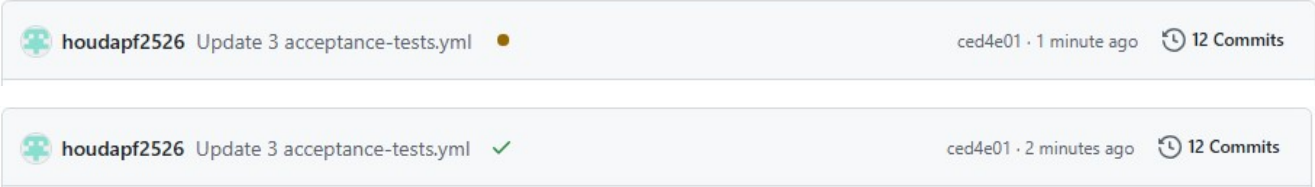
Pour valider ce TP, vous devez suivre rigoureusement les étapes suivantes :

- Ouvrez un terminal Git Bash et clonez le dépôt de référence sur votre machine :
```
git clone git@github.com:pf2026/site_vente_en_ligne.git
cd site_vente_en_ligne
```
- Ouvrez ensuite le projet dans votre IDE en tant que projet Maven.
- Modifiez le fichier `index.html` ou le fichier de style pour vous approprier le projet.
- Créez un nouveau dépôt public vide sur votre compte GitHub (ex: tp-test-nom-prenom).
- Changez l'adresse distante (Remote) de votre projet local pour pointer vers votre nouveau dépôt : `git remote set-url origin [URL_DE_VOTRE_NOUVEAU_REPO]`
- Envoyez le code (Push)
- Vérification de l'automatisation : Consultez votre compte GitHub, dans l'onglet "Actions". Vous devriez voir le workflow se déclencher. Un succès (icône verte) confirme que votre environnement de test est correctement configuré et automatisé.

4. Annexe : Preuves d'Exécution et de Conformité

Capture d'écran de la page index.html ouverte dans un navigateur local

Capture d'écran de l'onglet **"Actions"** sur le dépôt GitHub



Capture d'écran de l'onglet **"Code"** sur le dépôt GitHub